SNOWFLAKE DATABASE:

Creating a warehouse called ECOM_WH

```
CREATE WAREHOUSE IF NOT EXISTS ECOM_WH
WAREHOUSE_SIZE = 'XSMALL'
AUTO_SUSPEND = 60
AUTO_RESUME = TRUE
INITIALLY_SUSPENDED = TRUE;
```

> Creating database and checking the schemas:

```
CREATE DATABASE IF NOT EXISTS ECOM_DB;

SHOW SCHEMAS IN ECOM_DB;

CREATE SCHEMA IF NOT EXISTS ECOM_DB.RAW;
```

By default there are 2 shemas:
1. Information schema (this contains all the meta data and things required for snowflake to work…). It is like a data-dictionary and it is read only
2. Public schema : this is just for use to get started, there is no unique purpose for it

> table creation statements… joined tables with foreign keys

```
CREATE TABLE IF NOT EXISTS ECOM_DB.RAW.SALES (
    SALE_ID VARCHAR(15) PRIMARY KEY,
    ORDER_ID VARCHAR(10),
    PRODUCT_ID VARCHAR(10),
    PRODUCT_NAME VARCHAR(100),
    QUANTITY NUMBER(10,0),
    UNIT_PRICE NUMBER(12,2),
    DISCOUNT_PCT NUMBER(5,2),
    SALES_AMOUNT NUMBER(12,2),

    FOREIGN KEY (ORDER_ID)
        REFERENCES ECOM_DB.RAW.ORDERS(ORDER_ID)
);
```

```sql
CREATE TABLE IF NOT EXISTS ECOM_DB.RAW.CUSTOMER (
    CUSTOMER_ID VARCHAR(10) PRIMARY KEY,
    CUSTOMER_NAME VARCHAR(100),
    CUSTOMER_TYPE VARCHAR(50),
    CITY VARCHAR(50),
    STATE VARCHAR(50),
    EMAIL VARCHAR(100),
    SIGNUP_DATE DATE
);

CREATE TABLE IF NOT EXISTS ECOM_DB.RAW.ORDERS (
    ORDER_ID VARCHAR(10) PRIMARY KEY,
    CUSTOMER_ID VARCHAR(10),
    ORDER_DATE DATE,
    ORDER_CITY VARCHAR(50),
    SALES_CHANNEL VARCHAR(30),
    ORDER_STATUS VARCHAR(30),
    ORDER_TOTAL NUMBER(12,2),

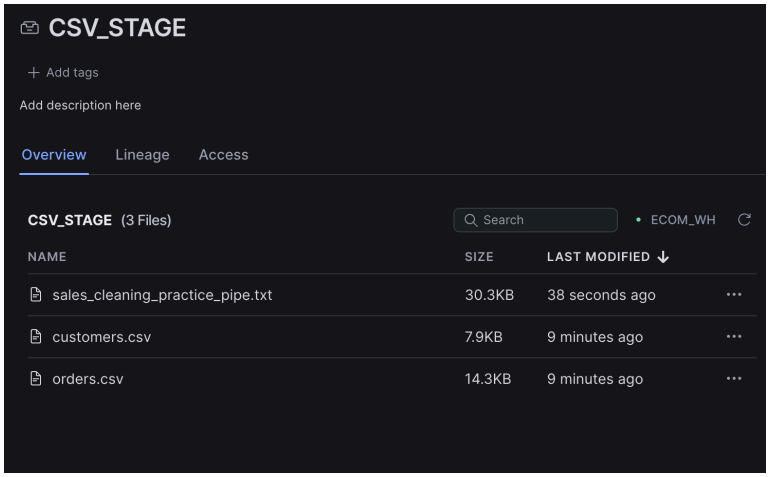
    FOREIGN KEY (CUSTOMER_ID)
        REFERENCES ECOM_DB.RAW.CUSTOMER(CUSTOMER_ID)
);
```

> Created a generic file format that works for 3 files, and created a stage that works with all the 3 files… so i will have only one stage where i will upload the files then use the copy into statement to load data

```sql
CREATE FILE FORMAT IF NOT EXISTS ECOM_DB.RAW.CSV_FILE
TYPE= CSV
SKIP_HEADER=1
FIELD_OPTIONALLY_ENCLOSED_BY = '"'
EMPTY_FIELD_AS_NULL = TRUE;

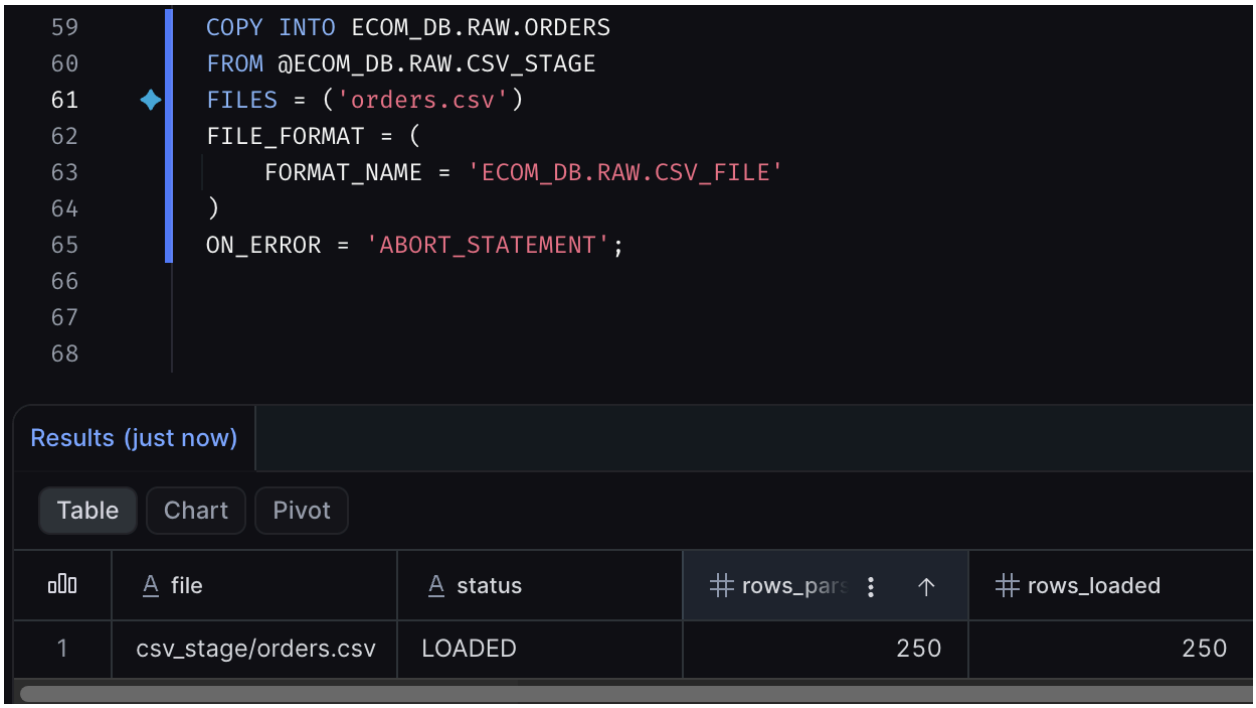
CREATE STAGE IF NOT EXISTS ECOM_DB.RAW.CSV_STAGE
FILE_FORMAT = ECOM_DB.RAW.CSV_FILE;
```

> In the stage while creating we told that file format is of type CSV but still we can upload the files!



We can see that i was able to upload .txt and .csv in the same stage
The problem comes when we use “COPY INTO” as the stage’s file format will be used..
To overcome this we can explicitly mention the file format to override the stage’s file format as shown below..(below are the 3 scenarios)

Scenario1: Use with mentioning the file format in the COPY INTO statement (the general one)



Scenario 2: we don’t mention file format as the snowflake will directly take it from the stage (which is default)

```
66
67  COPY INTO ECOM_DB.RAW.CUSTOMER
68  FROM @ECOM_DB.RAW.CSV_STAGE
69  FILES = ('customers.csv')
70  ON_ERROR = 'ABORT_STATEMENT';
71
```

Results (just now)

Table | Chart | Pivot

	file	status	rows_parsed	rows_loaded
1	csv_stage/customers.c	LOADED	100	100

Scenario 3: now we have to process our text file which has ‘|’ as delimiter
For this we will use the same stage and but we will explicitly mention the file format

```
73  COPY INTO ECOM_DB.RAW.SALES
74  FROM @ECOM_DB.RAW.CSV_STAGE
75  FILES = ('sales_cleaning_practice_pipe.txt')
76  FILE_FORMAT = (
77      FIELD_DELIMITER = '|'
78      SKIP_HEADER = 1
79      FIELD_OPTIONALLY_ENCLOSED_BY = '"'
80      EMPTY_FIELD_AS_NULL = TRUE
81  )
82  ON_ERROR = 'ABORT_STATEMENT';
83
84
85
```

Results (just now)

Table | Chart | Pivot

	file	status	rows_parsed	rows_loaded
1	csv_stage/sales_cleani	LOADED	645	645

We can see that we were able to load the data… so we have overridden the default file format and we used our new file format…

> Checking no of records using UNION ALL

```
SELECT 'CUSTOMER' AS TABLE_NAME, COUNT(*) AS TOTAL_RECORDS
FROM ECOM_DB.RAW.CUSTOMER

UNION ALL

SELECT 'ORDERS', COUNT(*)
FROM ECOM_DB.RAW.ORDERS

UNION ALL

SELECT 'SALES', COUNT(*)
FROM ECOM_DB.RAW.SALES;
```

Results (just now)

Table | Chart | Pivot — 3 rows — 403ms

	TABLE_NAME	TOTAL_RECORDS
1	CUSTOMER	100
2	ORDERS	250
3	SALES	645

> checking null values count

```
SELECT
    COUNT_IF(CUSTOMER_NAME IS NULL) AS NULL_NAMES,
    COUNT_IF(CUSTOMER_TYPE IS NULL) AS NULL_TYPES,
    COUNT_IF(CITY IS NULL) AS NULL_CITIES,
    COUNT_IF(EMAIL IS NULL) AS NULL_EMAILS
FROM ECOM_DB.RAW.CUSTOMER;

SELECT
    COUNT_IF(CUSTOMER_ID IS NULL) AS NULL_CUSTOMER_IDS,
    COUNT_IF(ORDER_CITY IS NULL) AS NULL_CITIES,
    COUNT_IF(ORDER_STATUS IS NULL) AS NULL_STATUSES
FROM ECOM_DB.RAW.ORDERS;

SELECT
    COUNT_IF(PRODUCT_NAME IS NULL) AS NULL_PRODUCT_NAMES,
    COUNT_IF(QUANTITY IS NULL) AS NULL_QUANTITIES,
    COUNT_IF(SALES_AMOUNT IS NULL) AS NULL_SALES
FROM ECOM_DB.RAW.SALES;
```

Results (just now)

Table | Chart | Pivot — 1 row — 86ms

	NULL_PRODUCT_NAMES	NULL_QUANTITIES	NULL_SALES
1	4	0	0

>cleaning

```sql
CREATE OR REPLACE TABLE ECOM_DB.RAW.CUSTOMER_CLEAN AS
SELECT
    CUSTOMER_ID,
    COALESCE(UPPER(TRIM(CUSTOMER_NAME)), 'UNKNOWN')
        AS CUSTOMER_NAME,
    COALESCE(UPPER(TRIM(CUSTOMER_TYPE)), 'UNCLASSIFIED')
        AS CUSTOMER_TYPE,
    COALESCE(UPPER(TRIM(CITY)), 'UNKNOWN')
        AS CITY,
    COALESCE(UPPER(TRIM(STATE)), 'UNKNOWN')
        AS STATE,
    COALESCE(LOWER(TRIM(EMAIL)), 'NOT PROVIDED')
        AS EMAIL,
    CAST(SIGNUP_DATE AS DATE) AS SIGNUP_DATE
FROM ECOM_DB.RAW.CUSTOMER;
```

Results (just now)

Table | Chart | Pivot

	status
1	Table CUSTOMER_CLEAN successfully created.

> Analytical Queries: totals

```sql
SELECT
    C.CUSTOMER_TYPE,
    COUNT(DISTINCT C.CUSTOMER_ID) AS TOTAL_CUSTOMERS,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    SUM(S.SALES_AMOUNT) AS TOTAL_REVENUE
FROM ECOM_DB.RAW.CUSTOMER_CLEAN C
JOIN ECOM_DB.RAW.ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
JOIN ECOM_DB.RAW.SALES S
    ON O.ORDER_ID = S.ORDER_ID
GROUP BY C.CUSTOMER_TYPE
ORDER BY TOTAL_REVENUE DESC;
```

Results (just now)

Table | Chart | Pivot — 5 rows — 297ms

	CUSTOMER_TYPE	TOTAL_CUSTOMERS	TOTAL_ORDERS	TOTAL_REVENUE
1	RETAIL	45	125	2737963.65
2	SMALL BUSINESS	21	57	1399649.70
3	WHOLESALE	15	37	646017.60
4	CORPORATE	11	25	464558.05
5	UNCLASSIFIED	1	6	308215.70

> actual query

```
SELECT
    O.ORDER_CITY,
    SUM(S.SALES_AMOUNT) AS TOTAL_REVENUE
FROM ECOM_DB.RAW.ORDERS O
JOIN ECOM_DB.RAW.SALES S
    ON O.ORDER_ID = S.ORDER_ID
GROUP BY O.ORDER_CITY;
```

Results (just now)

Table | Chart | Pivot | Next steps | 22 rows | 341ms

	ORDER_CITY	TOTAL_REVENUE
	Ahmedabad 4.5% Chennai 4.5% +19 more	3597 – 804671.85
1	Jaipur	804671.85
2	CHENNAI	84641.50
3	Ahmedabad	542347.00
4	Hyderabad	3597.00
5	Pune	406834.00
6	Delhi	501248.30
7	Kolkata	485845.10

> optimized by using with keyword

```
WITH SALES_AGG AS (
    SELECT
        ORDER_ID,
        SUM(SALES_AMOUNT) AS TOTAL_REVENUE
    FROM ECOM_DB.RAW.SALES
    GROUP BY ORDER_ID
)
SELECT
    O.ORDER_CITY,
    SUM(S.TOTAL_REVENUE) AS TOTAL_REVENUE
FROM ECOM_DB.RAW.ORDERS O
JOIN SALES_AGG S
    ON O.ORDER_ID = S.ORDER_ID
GROUP BY O.ORDER_CITY;
```

Results (just now)

Table | Chart | Pivot | Next steps | 22 rows | 312ms

	ORDER_CITY	TOTAL_REVENUE
	Ahmedabad 4.5% Chennai 4.5% +19 more	3597 – 804671.85
1	Jaipur	804671.85
2	CHENNAI	84641.50
3	Ahmedabad	542347.00
4	Hyderabad	3597.00
5	Pune	406834.00
6	Delhi	501248.30
7	Kolkata	485845.10